

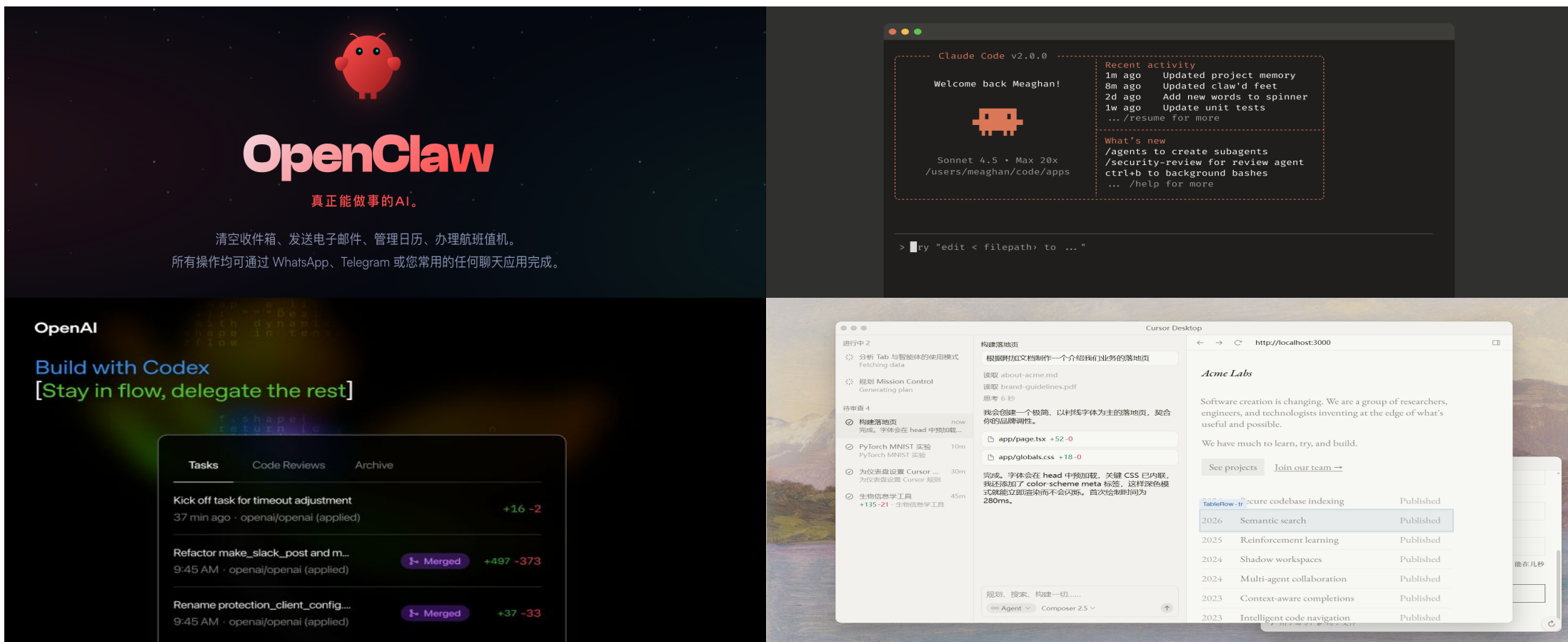
# From MHA to DSA: The Evolution of Attention Paradigms in Long-Context LLMs

A Deep Dive into Low-Rank Compression,  
Sparse Attention of DeepSeek-V4

Huang Xiaotong, 2026/06/05

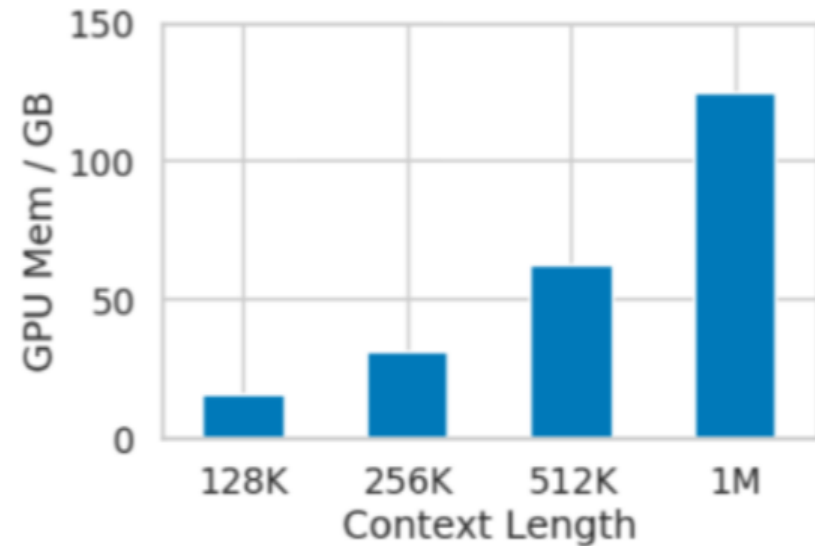
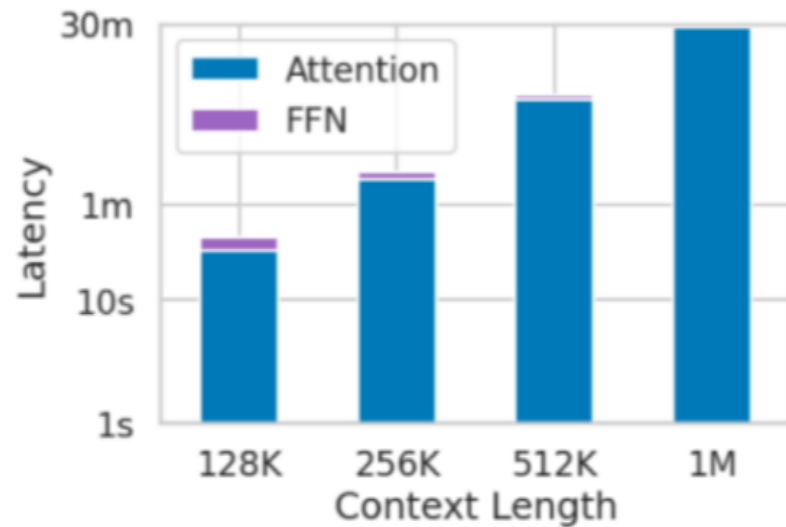
# The Transition to Million-Token Context Intelligence

Next-generation applications require processing **ultra-long contexts**—ranging from complex agentic workflows to multi-document analysis.



# Long-Context LLMs Inference is expensive

The scaling paradigm is fundamentally constrained by the  $\mathcal{O}(N^2)$  complexity.



**Long Prefilling Latency:** 30 minutes to process 1M tokens on an A100 for an 8B LLM.

**Large GPU Memory cost:** over 100 GB memory is required for 1M tokens in fp16.

# Prerequisite: KV Cache

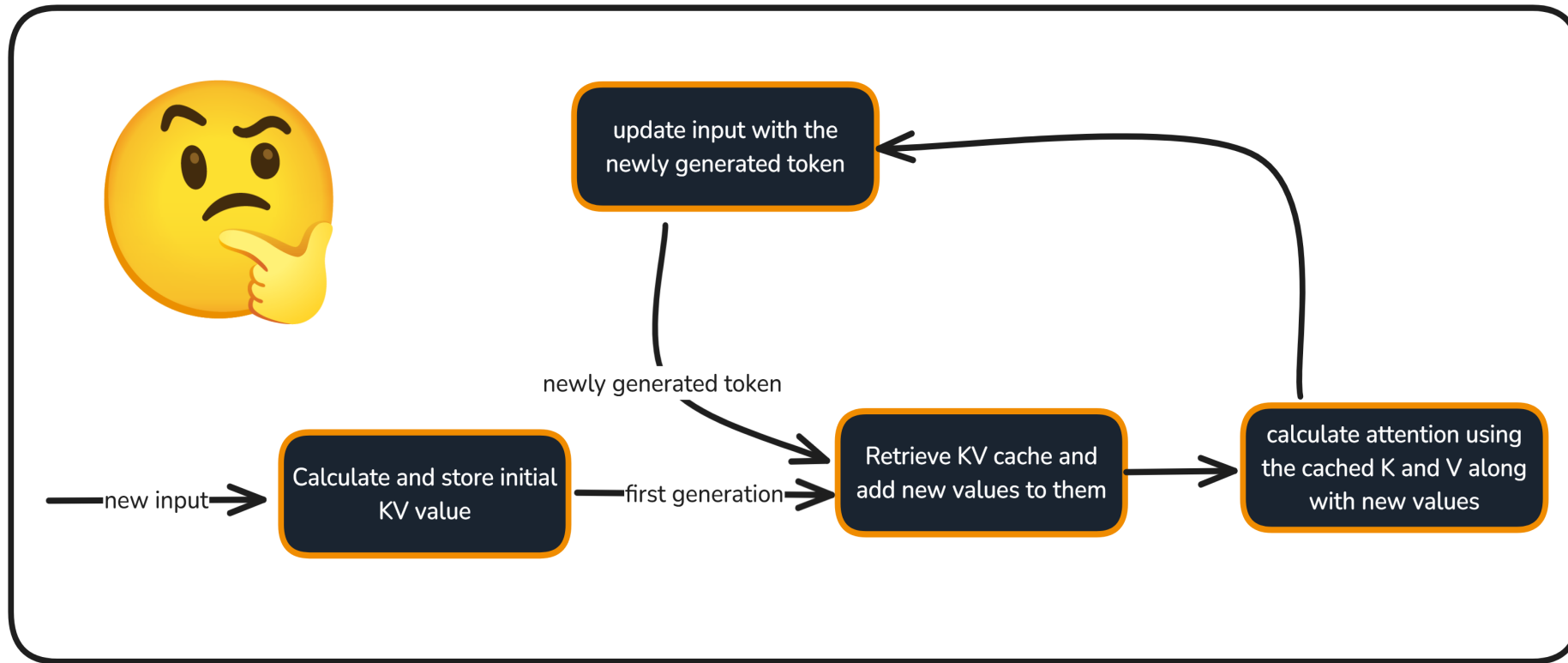
---

## Why KV Cache?

- Causal Autoregressive Inference
- Cumulative Computational Redundancy

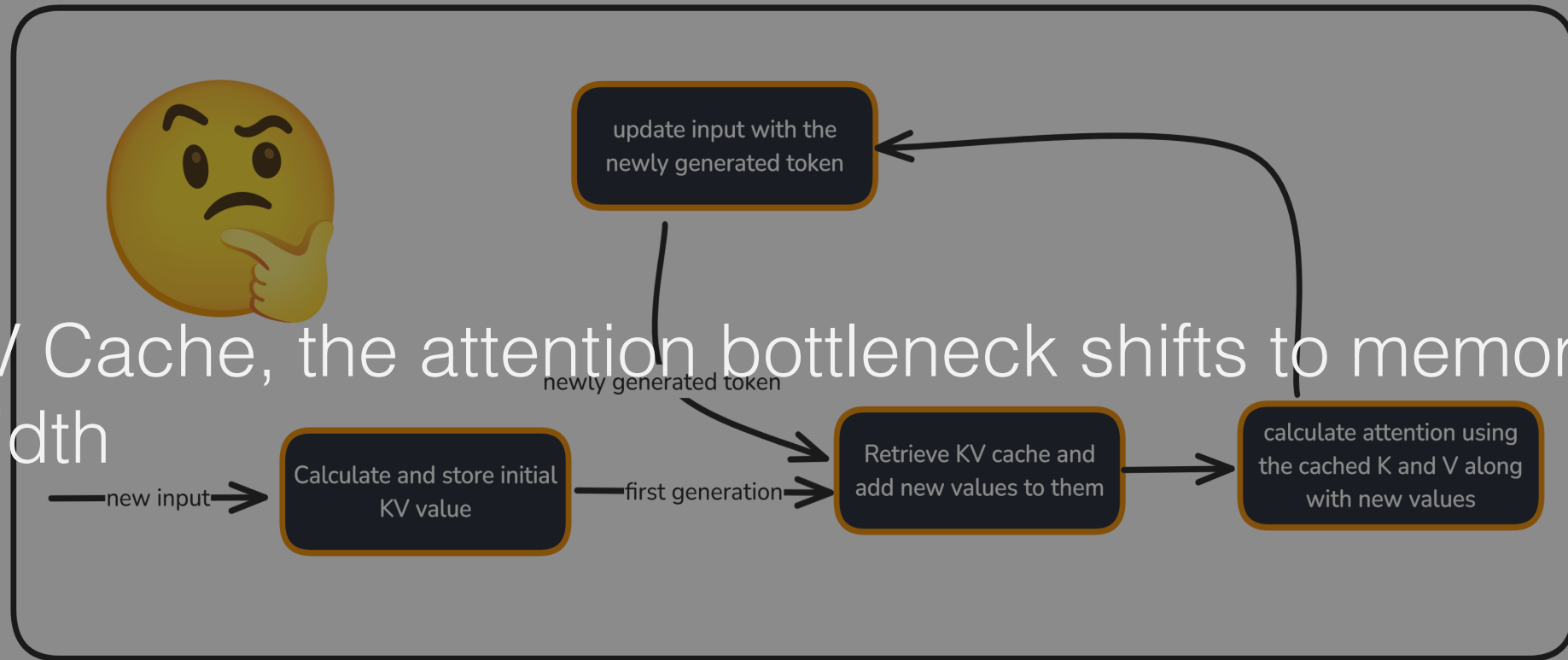
|       |       |       |       |      |  |   |
|-------|-------|-------|-------|------|--|---|
|       |       | 4     |       |      |  |   |
| <bos> | x     | -inf  | -inf  | -inf |  |   |
| Hello | x     | x     | -inf  | -inf |  |   |
| World | x     | x     | x     | -inf |  | 4 |
| !     | x     | x     | x     | x    |  |   |
|       | <bos> | Hello | world | !    |  |   |

# Prerequisite: KV Cache



Instead of re-computing historical context, the  $K$  and  $V$  matrices of past tokens are computed **exactly once** and permanently stored in memory.

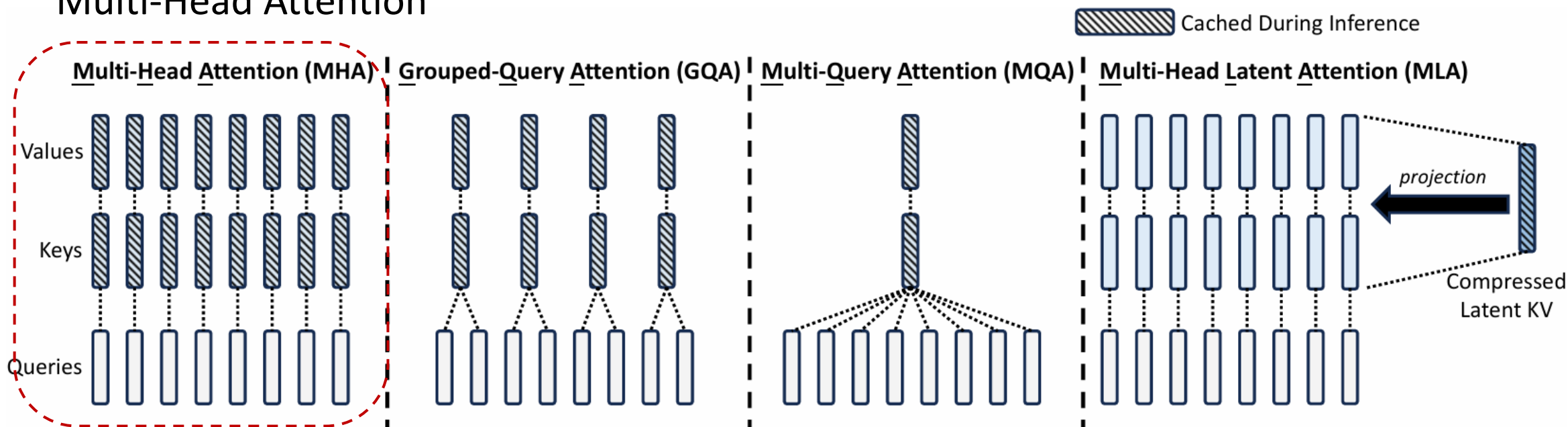
# Prerequisite: KV Cache



Instead of re-computing historical context, the  $K$  and  $V$  matrices of past tokens are computed **exactly once** and permanently stored in memory.

# The Evolution: Squeezing the Head Dimension

## Multi-Head Attention



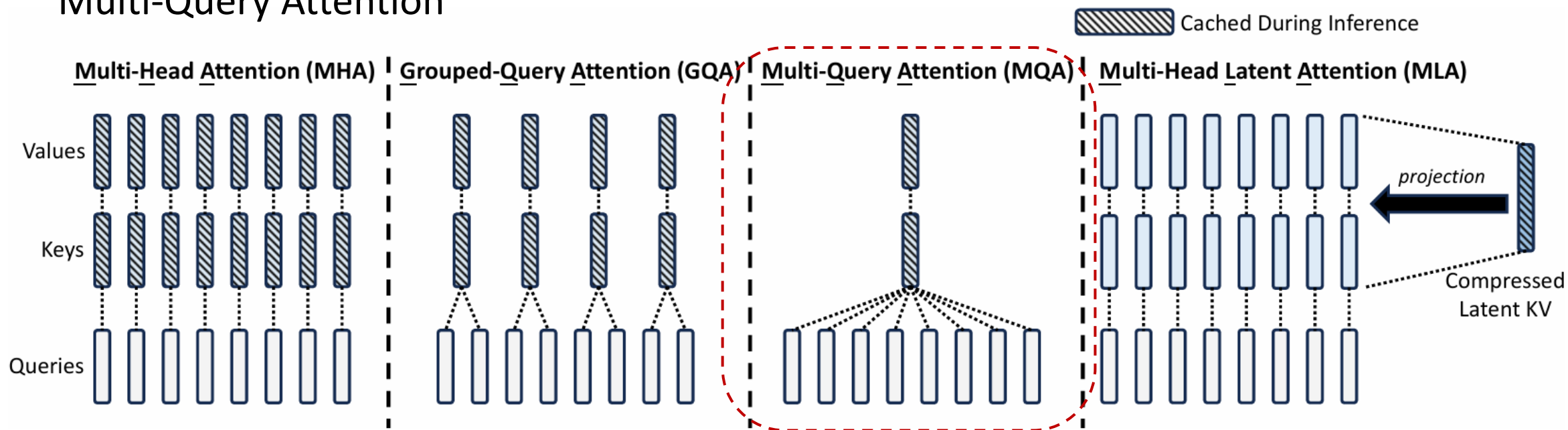
MHA: Every individual Query head is paired with a **unique** Key and Value head (  $H_Q = H_K = H_V$  ) .

✓ Maximum representational capacity.

✗ Significantly be constrained to memory bandwidth.

# The Evolution: Squeezing the Head Dimension

## Multi-Query Attention



MQA: All Query heads share a **single** Key and Value head (  $H_K = H_V = 1$  ) .

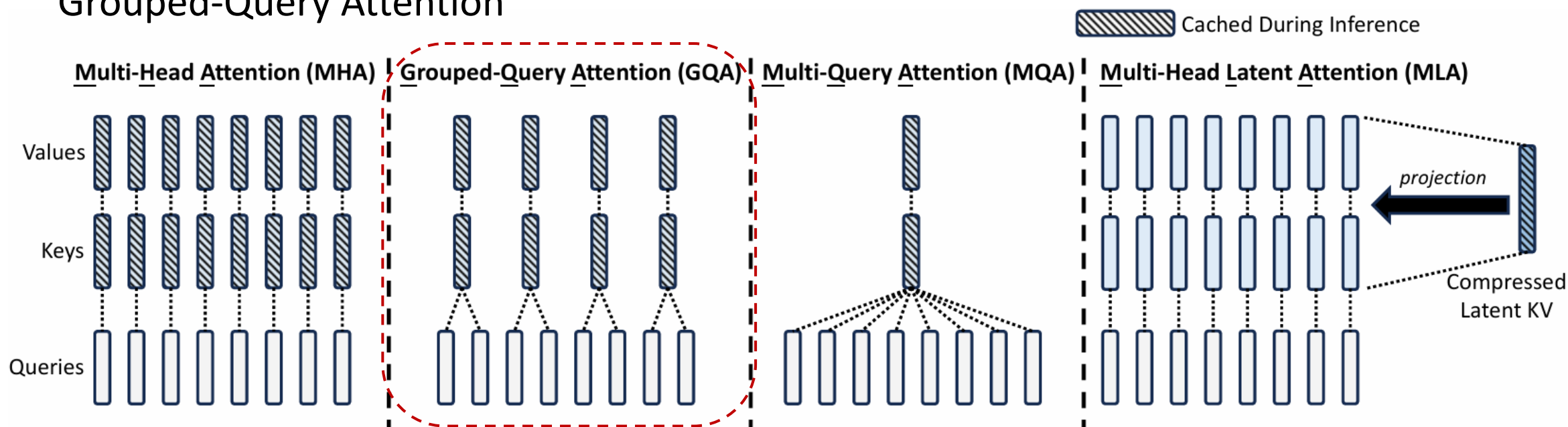
✓ Reduce HBM bandwidth pressure.

✗ Degradation in model expressiveness.



# The Evolution: Squeezing the Head Dimension

## Grouped-Query Attention



GQA: Query heads are partitioned into **groups**; each group shares a single local Key and Value head ( $1 < H_K = H_V < H_Q$ ).

✓ Better representational capacity than MQA.

✗ More Complicated implementation and more memory bandwidth.

# DeepSeek V2: Multi-Head Latent Attention

---

## Down/Up Projection

- Down-Projection Stage:  
MLA projects the high-dimensional hidden state  $h_t$  into a low-dimensional latent vector  $c_t^{KV}$  via a down-projection matrix  $W^{KV}$ .
- Up-Projection Stage:  
During the decode phase, the latent vector is projected back to the head space using learned up-projection matrices  $W^{UK}$  and  $W^{UV}$ .

$$\begin{array}{ll} \text{MHA:} & \\ (1, d_m) & (h, d) \\ \mathbf{q}_t = W^Q \mathbf{h}_t, & \\ \mathbf{k}_t = W^K \mathbf{h}_t, & \\ \mathbf{v}_t = W^V \mathbf{h}_t, & \end{array} \quad \longrightarrow \quad \begin{array}{ll} \text{MLA:} & (1, d_c) \\ \mathbf{c}_t^Q = W^{DQ} \mathbf{h}_t, & \\ \mathbf{q}_t^C = W^{UQ} \mathbf{c}_t^Q, & \\ \mathbf{c}_t^{KV} = W^{DKV} \mathbf{h}_t, & \\ \mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV}, & \\ \mathbf{v}_t^C = W^{UV} \mathbf{c}_t^{KV}, & \end{array}$$

---

Motivated by LoRA, through caching only  $c_t^{KV}$  instead of full-head KV tensors, the memory footprint is drastically reduced from  $O(s, h, d)$  to  $O(s, d_c)$ .

# DeepSeek V2: Multi-Head Latent Attention

---

## Matrix Absorption

QK Absorption:

$$\underbrace{q_t^{C\top} k_t^C = (W^{UQ} c_t^Q)^\top W^{UK} c_t^{KV}}_{\text{传统计算方式}} = \underbrace{c_t^{Q\top} W^{UQ\top} W^{UK} c_t^{KV}}_{\text{矩阵结合律}} = c_t^{Q\top} \boxed{W^{UQ\top} W^{UK}} c_t^{KV} = c_t^{Q\top} W c_t^{KV}$$

VO Absorption:

$$(p \cdot (c_{kv} \cdot W^{UV})) \cdot W^O = (p \cdot c_{kv}) \cdot (W^{UV} \cdot W^O) = (\text{softmax}(q_{nope} \cdot c_{kv} + q_{pe} \cdot k_{pe}) \cdot c_{kv}) \cdot \boxed{W^{UV} \cdot W^O}$$

---

The "Matrix Absorption" principle allows MLA to bypass the explicit reconstruction of full-head Key (K) and Value (V) matrices during inference.

# DeepSeek V2: Multi-Head Latent Attention

---

## DeepSeek V2: Decoupled RoPE

w/o RoPE:

$$q_{t,i}^T \times k_{j,i} = (W_{(i)}^{UQ} c_t^Q)^T \times W_{(i)}^{UK} c_j^{KV} = (c_t^Q)^T \times (W_{(i)}^{UQ})^T W_{(i)}^{UK} \times c_j^{KV}$$

w RoPE:

$$\begin{aligned} q_{t,i}^T \times k_{j,i} &= (\mathcal{R}_t W_{(i)}^{UQ} c_t^Q)^T \times \mathcal{R}_j W_{(i)}^{UK} c_j^{KV} \\ &= (c_t^Q)^T \times (W_{(i)}^{UQ})^T \boxed{\mathcal{R}_{t-j}} W_{(i)}^{UK} \times c_j^{KV} \end{aligned}$$

Unable to  
absorb !

$$(p \cdot (c_{kv} \cdot W^{UV})) \cdot W^O = (p \cdot c_{kv}) \cdot (W^{UV} \cdot W^O) = (\text{softmax}(q_{nope} \cdot c_{kv} + \boxed{q_{pe} \cdot k_{pe}}) \cdot c_{kv}) \cdot W^{UV} \cdot W^O$$

# DeepSeek V2: Multi-Head Latent Attention

---

## Comparison

| Mechanism      | KV Cache / Token | Operations(Decode) | Expressivity |
|----------------|------------------|--------------------|--------------|
| MHA            |                  |                    | High         |
| GQA(8:1 Group) |                  |                    | Reduced      |
| MQA            |                  |                    | Lowest       |
| MLA            |                  |                    | Almost High  |

---

Key Takeaway: MLA reduces significantly KV Cache size by trading off **more computation operations**. ( $d < d_c < h \times d$ )

# What We Have vs. What Remains.

---

## From MLA to V4's Hybrid Attention

- KV Cache: storing  $n$  tokens requires caching  $n$  key-value (KV) pairs for an  $n$ -length context.
- Bandwidth Bottleneck: When generating the  $t$ -th token, all preceding  $t - 1$  KV pairs must be read for attention.
- MLA: Multi-head Latent Attention compresses KV into a low-rank latent variable  $c_t^{KV} \in \mathbb{R}^{d_c}$ , storing only one compressed vector per token.
- Remaining Challenge: MLA reduces the width but remains the number of KV entries (sequence length  $n$ ).

# DeepSeek V4: Hybrid Attention

---

## Compress Sparse Attention

(CSA)

- Merges every 4 tokens into 1 entry.
- To further reduce computation, it applies DeepSeek Sparse Attention (DSA), where each query attends to only the top-k compressed KV entries.

## Heavily Compress Attention

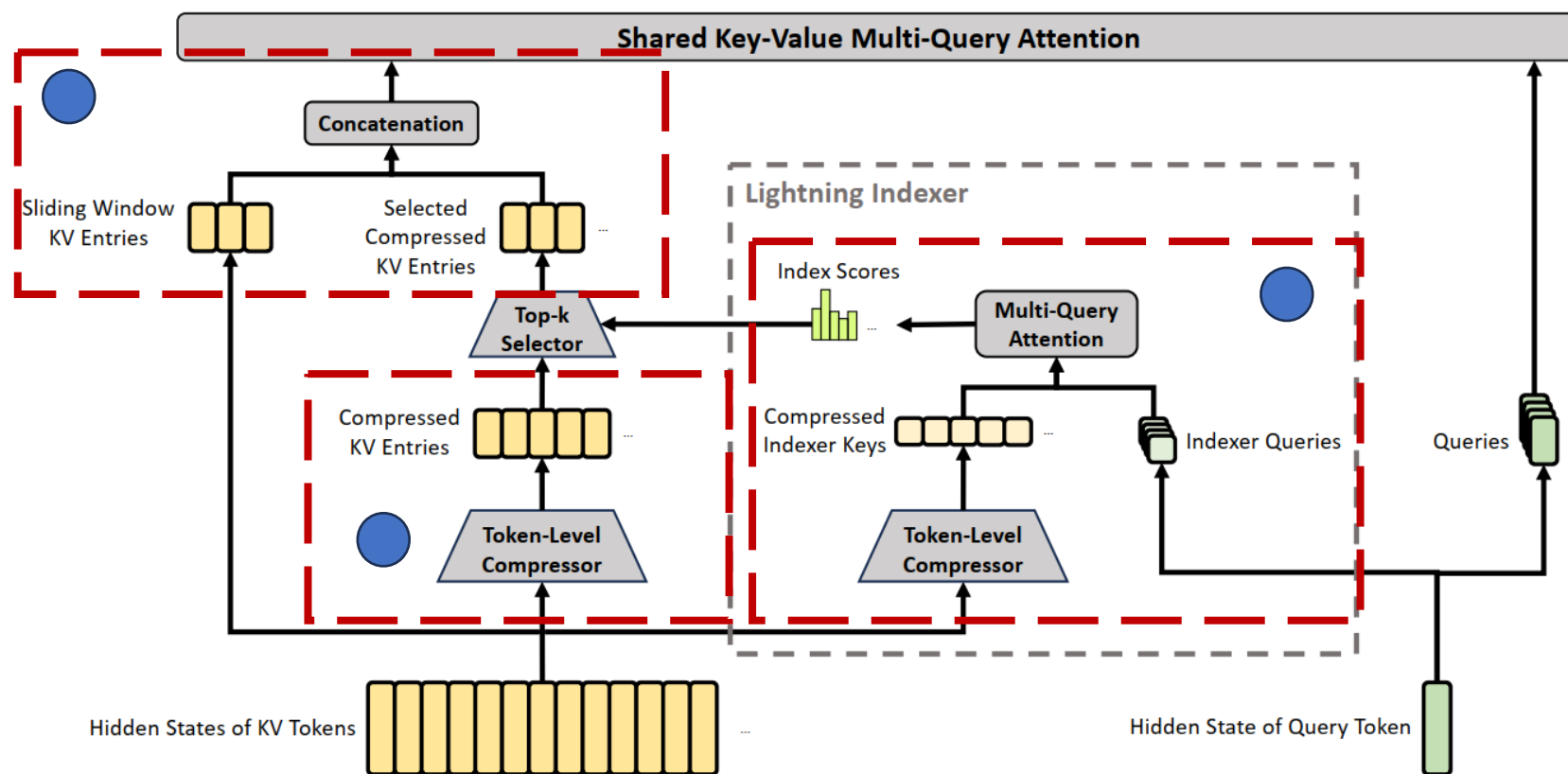
(HCA)

- Merges every 128 tokens into 1 entry.
- No sparsity is applied; attention is fully dense over the compressed entries.

CSA and HCA layers are interleaved.

# DeepSeek V4: Hybrid Attention

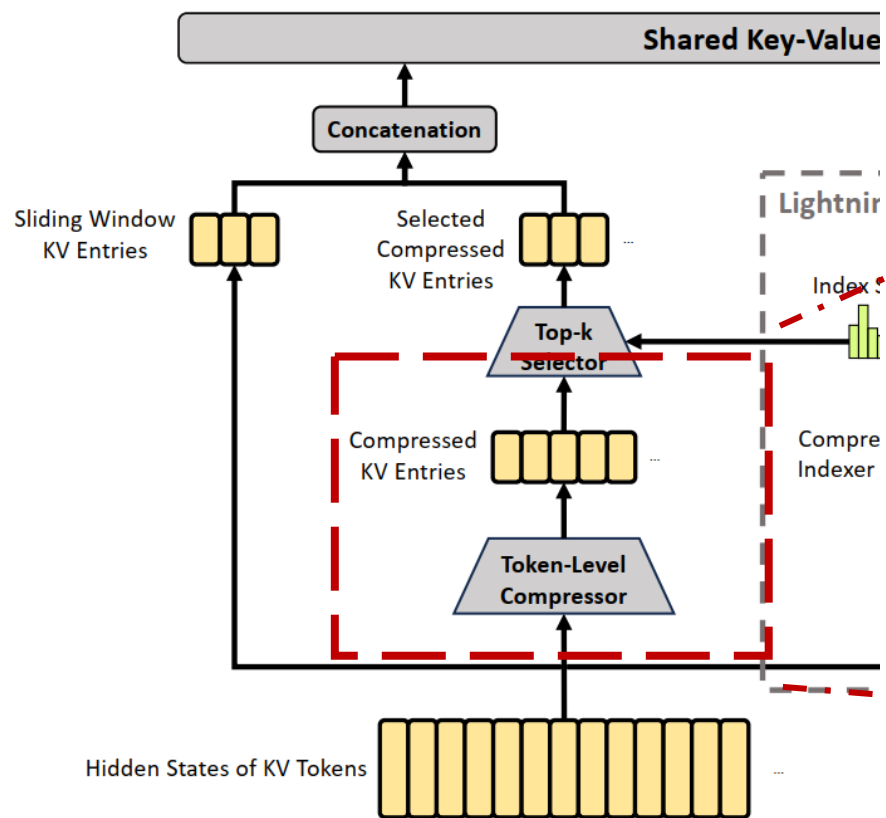
## Compress Sparse Attention





# DeepSeek V4: Hybrid Attention

## CSA: Token-Level Compressor



$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}$$

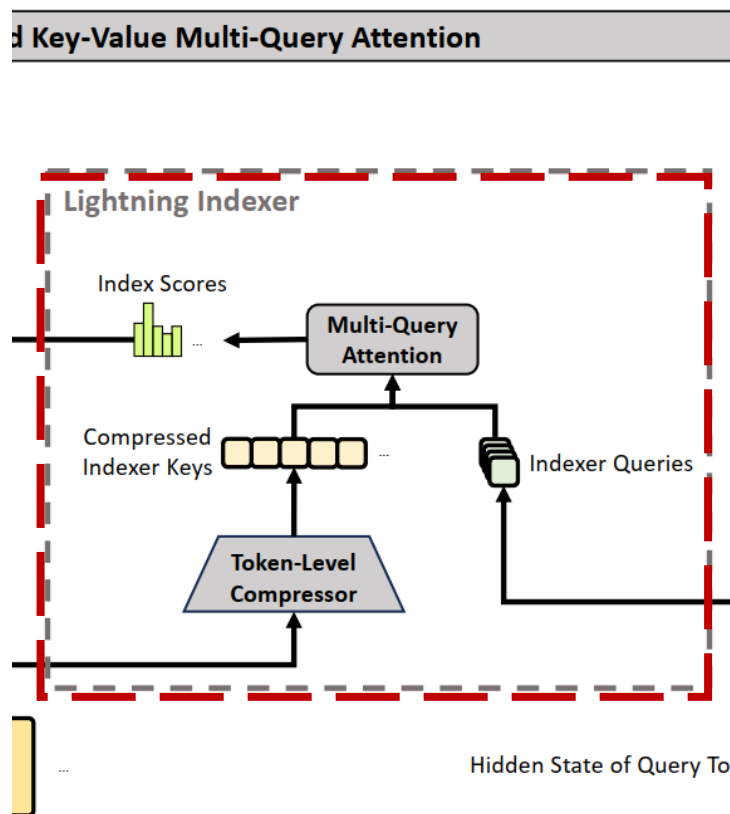
$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]),$$

$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b,$$

Note: When compressing KV entries, two adjacent compressed KV blocks share the original tokens but employ different projections.

# DeepSeek V4: Hybrid Attention

## CSA: Lightning Indexer



$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ},$$
$$[\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_h^I}^I] = \mathbf{q}_t^I = \mathbf{c}_t^Q \cdot W^{IUQ},$$

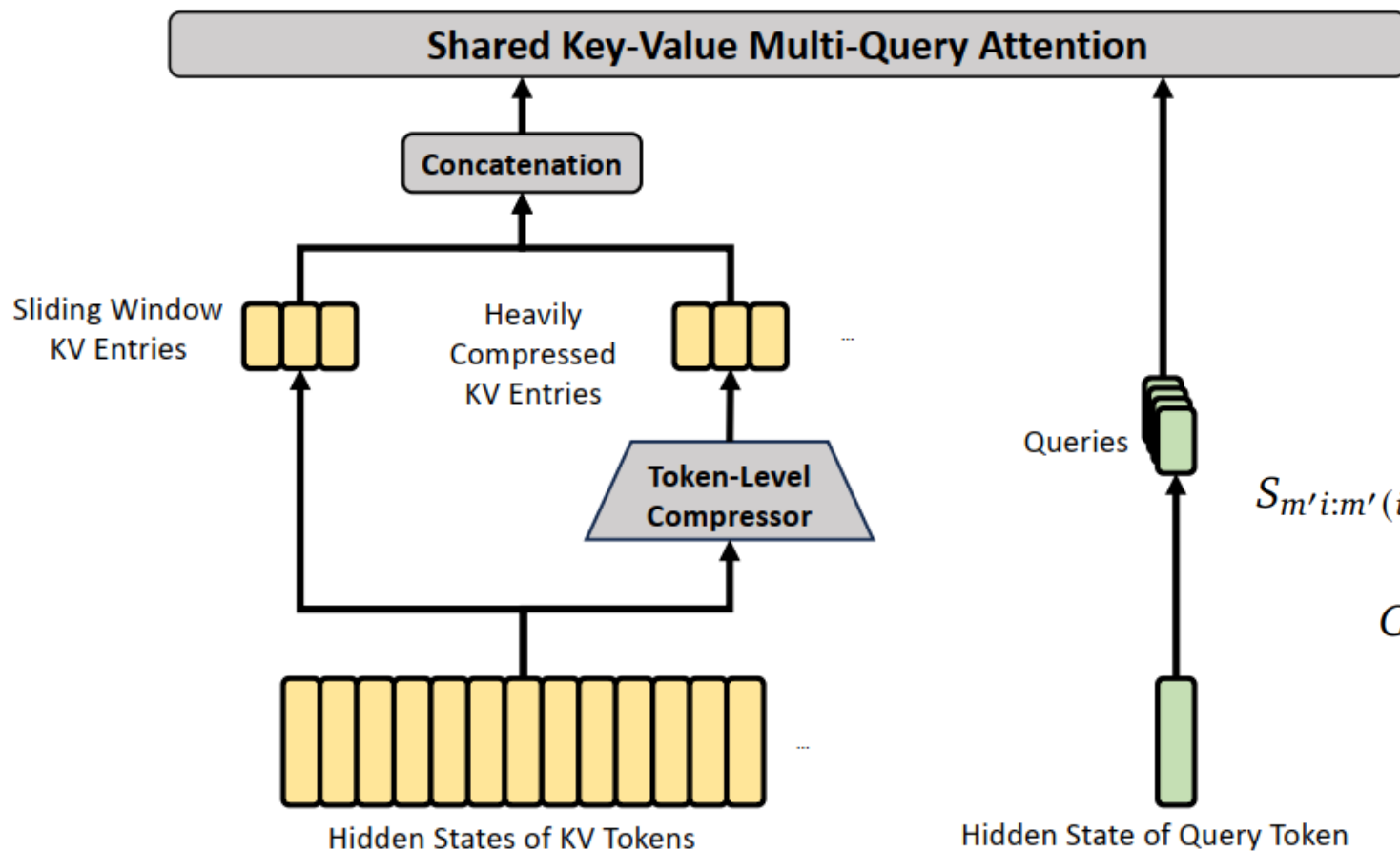
$$[w_{t,1}^I; w_{t,2}^I; \dots; w_{t,n_h^I}^I] = \mathbf{w}_t^I = \mathbf{h}_t \cdot W^w,$$

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \text{ReLU}(\mathbf{q}_{t,h}^I \cdot K_s^{\text{IComp}}),$$

The Indexer uses a shared-latent low-rank query combined with 64-head FP4 dot products to select the top-512 compressed KV entries for each query from a total of  $\frac{n}{m}$  entries.

# DeepSeek V4: Hybrid Attention

## Heavily Compress Attention



$$C = H \cdot W^{KV},$$

$$Z = H \cdot W^Z,$$

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m'i:m'(i+1)-1} + B),$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j.$$

# DeepSeek V4: Hybrid Attention

---

## Comparison for 1M token

BF16 GQA8 baseline

KV Cache per token =  $2 (\mathbf{K} + \mathbf{V}) \times 8 (\mathbf{heads}) \times 128 (\mathbf{heads\ dim}) \times 2 \text{ bytes } (\mathbf{BF16}) = 4 \text{ KB}$

1M token = 4 GB / layer

DeepSeek-V4-Pro

KV Cache per token =  $64 (\mathbf{RoPE\ dim}) \times 2 \text{ bytes } (\mathbf{BF16}) + 448 (\mathbf{heads\ dim}) \times 1 \text{ bytes } (\mathbf{FP8}) = 576 \text{ bytes}$

1M token for CSA layer =  $(1 \text{ M (entries)} / 4) \times 576 = 144 \text{ MB / layer}$

1M token for HCA layer =  $(1 \text{ M (entries)} / 128) \times 576 = 4.5 \text{ MB / layer}$

---

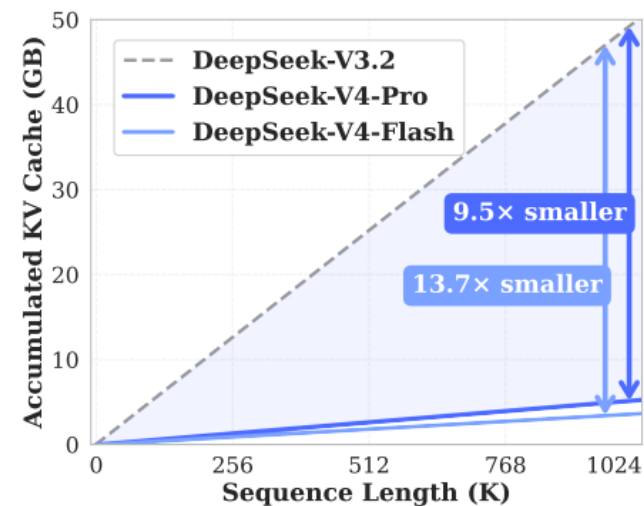
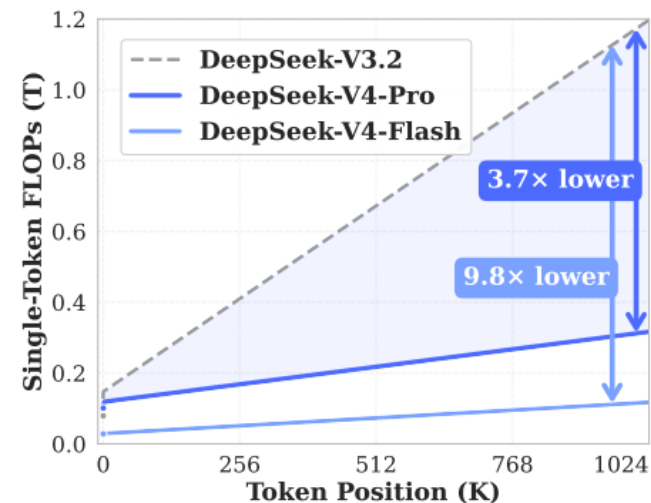
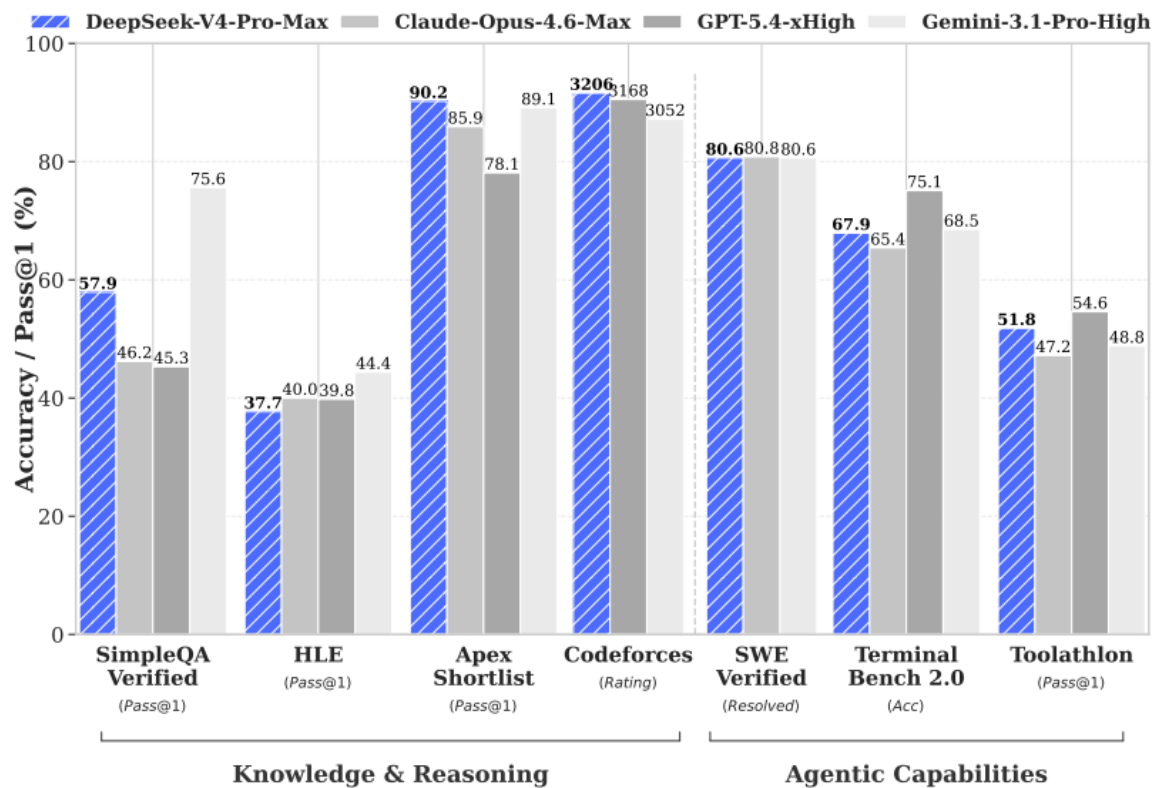
DeepSeek V4 performance gains from:

- Sequence compression (4x), lowering KV size per entry.
- Precision reduction (BF16 -> FP8/FP4)

Bandwidth reduction = compression ratio  $\times$  precision ratio.

# DeepSeek V4: Hybrid Attention

## Comparison with DeepSeek V3.2



# Conclusion

---

- MLA: Compress KV cache into low-dimensional latent vectors and recovers them on-the-fly.
- CSA / HCA: Packs tokens into semantic blocks to reduce the length of context.
- Auxiliary: Overlapping windows, mixed precision, decoupled RoPE...

## Takeaways

- Hierarchy system: coarse-to-fine (HCA, CSA, MLA), mixed precision (BF16 RoPE, FP8/FP4), global-to-local (lightning indexer)
- System-algorithm co-design: MLA: reduce entry size, CSA reduce number of entries..., memory-compute trade-off...